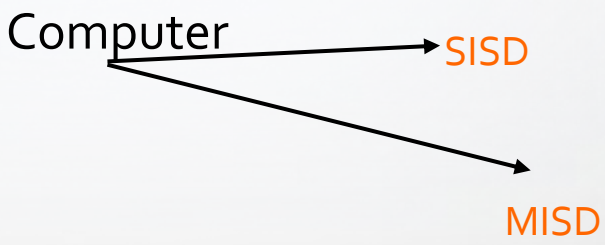
GPU ARCHITECTURE

Prof. V. Mele
Parallel High Performance Computing
University of Naples Federico II
a.a. 2025/26

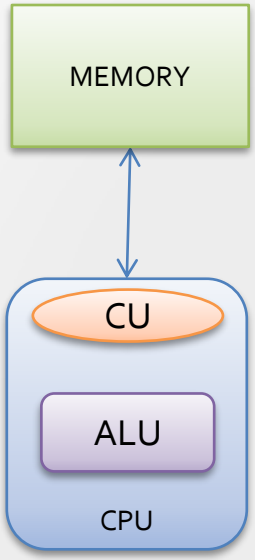
Flynn Taxonomy

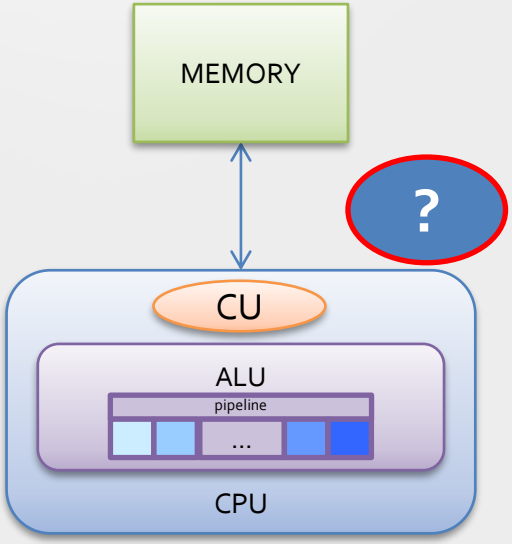


Instruction Stream: stream of instructions to execute.

Data Stream: stream of data to work on in the instruction execution.

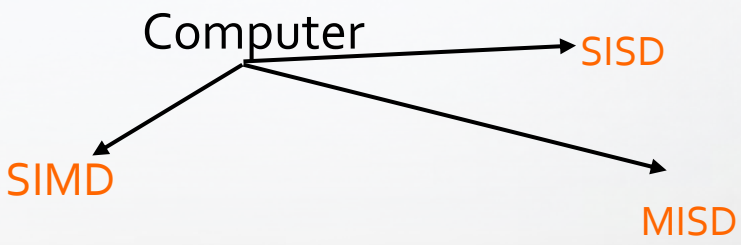
[1]





- Single Instruction Stream / Single Data Stream (SISD)
- Single Instruction Stream / Multiple Data Stream (SIMD)
- Multiple Instruction Stream / Single Data Stream (MISD)
- Multiple Instruction Stream / Multiple Data Stream (MIMD)

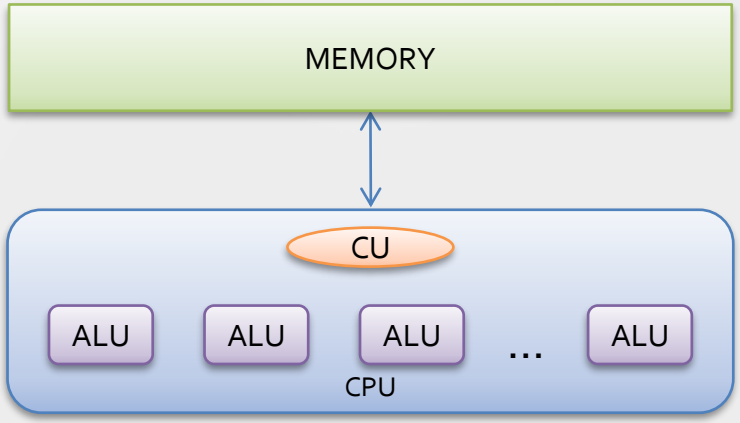
Flynn Taxonomy



Instruction Stream: stream of instructions to execute.

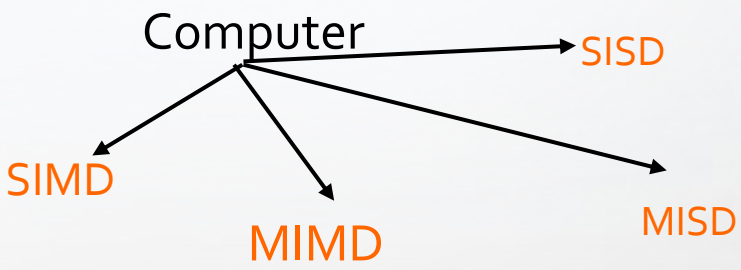
Data Stream: stream of data to work on in the instruction execution.

[1]



- Single Instruction Stream / Single Data Stream (SISD)
- Single Instruction Stream / Multiple Data Stream (SIMD)
- Multiple Instruction Stream / Single Data Stream (MISD)
- Multiple Instruction Stream / Multiple Data Stream (MIMD)

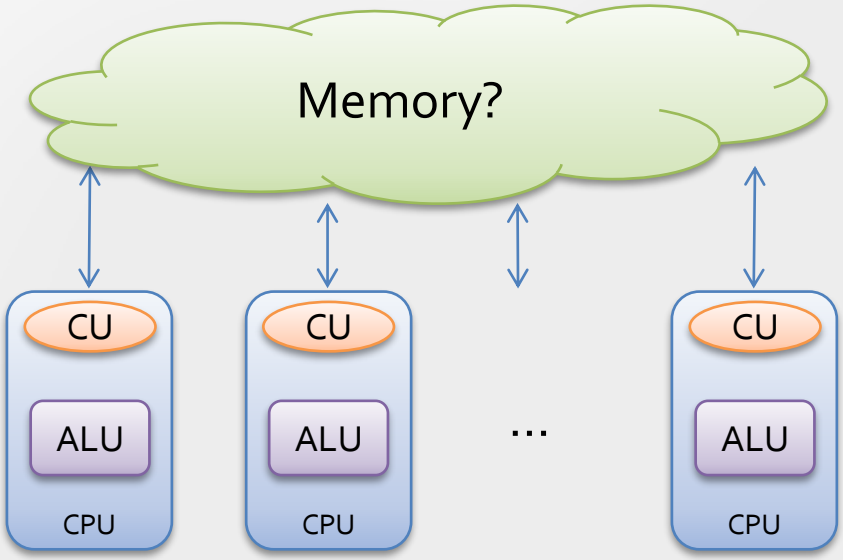
Flynn Taxonomy



Instruction Stream: stream of instructions to execute.

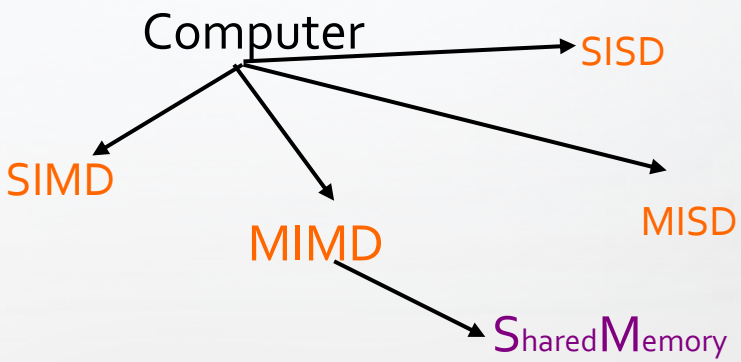
Data Stream: stream of data to work on in the instruction execution.

[1]



- Single Instruction Stream / Single Data Stream (SISD)
- Single Instruction Stream / Multiple Data Stream (SIMD)
- Multiple Instruction Stream / Single Data Stream (MISD)
- Multiple Instruction Stream / Multiple Data Stream (MIMD)

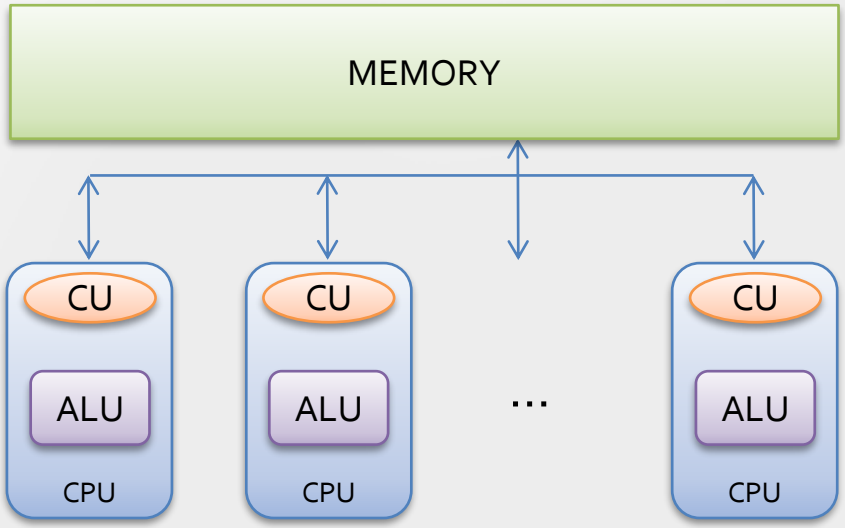
Flynn Taxonomy



Instruction Stream: stream of instructions to execute.

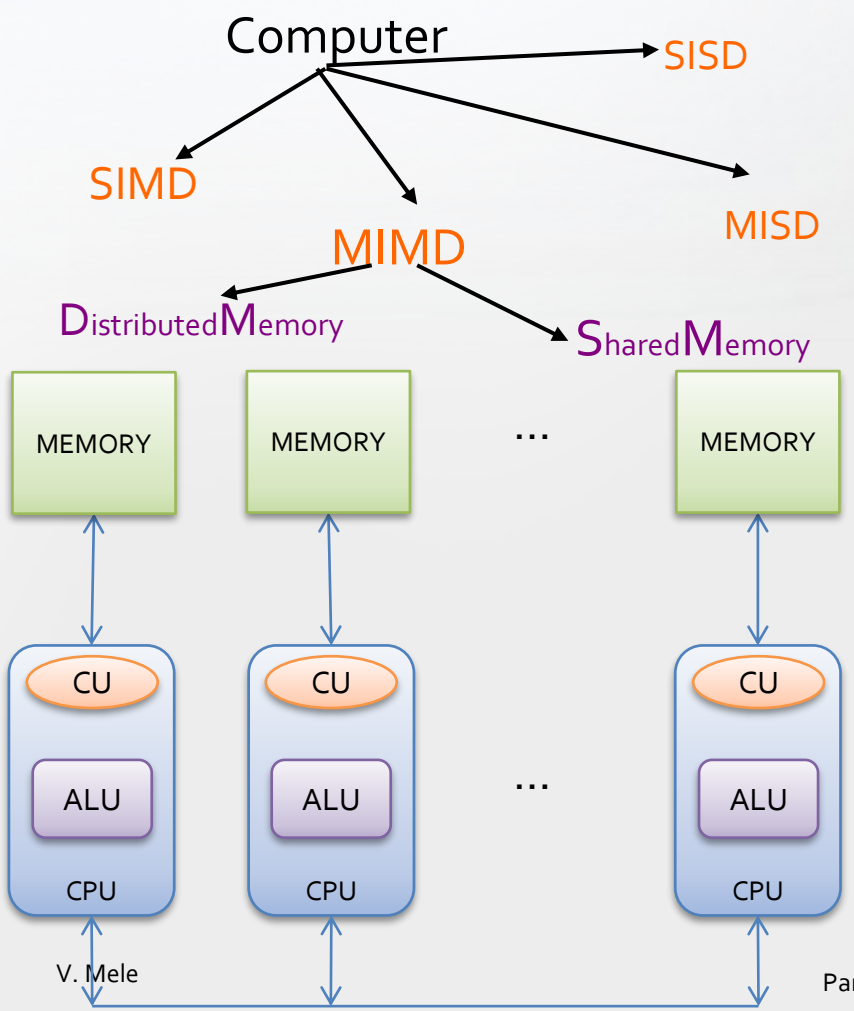
Data Stream: stream of data to work on in the instruction execution.

[1]



- Single Instruction Stream / Single Data Stream (SISD)
- Single Instruction Stream / Multiple Data Stream (SIMD)
- Multiple Instruction Stream / Single Data Stream (MISD)
- Multiple Instruction Stream / Multiple Data Stream (MIMD)

Flynn Taxonomy



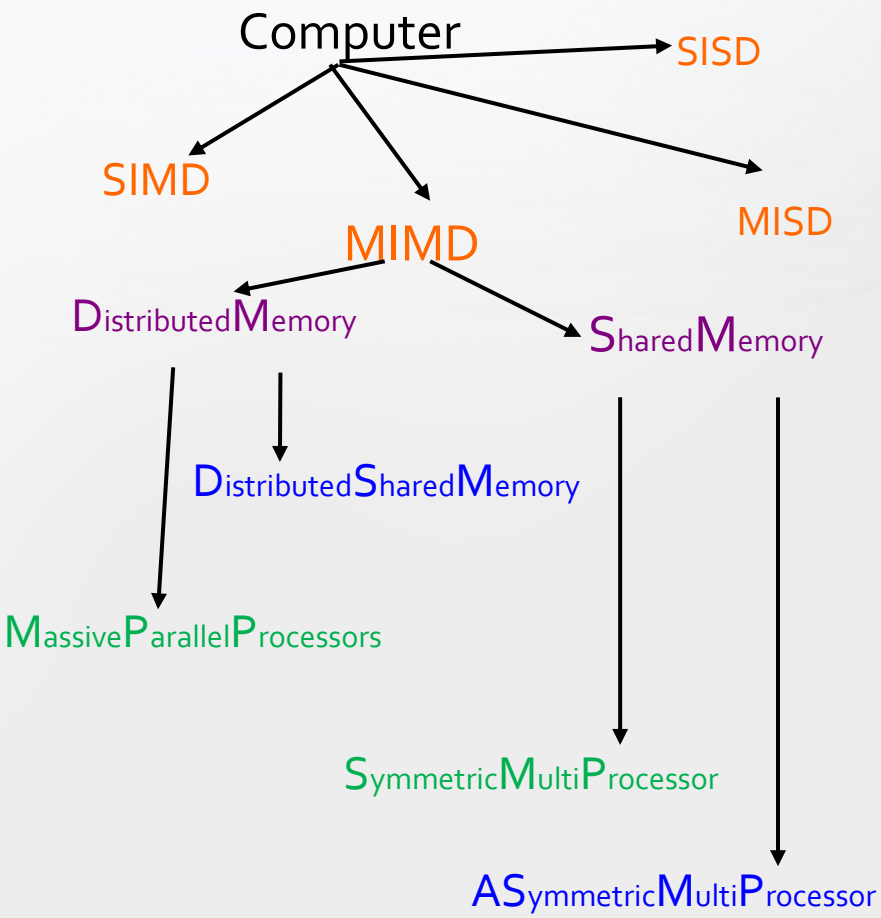
Instruction Stream: stream of instructions to execute.

Data Stream: stream of data to work on in the instruction execution.

[1]

- Single Instruction Stream / Single Data Stream (SISD)
- Single Instruction Stream / Multiple Data Stream (SIMD)
- Multiple Instruction Stream / Single Data Stream (MISD)
- Multiple Instruction Stream / Multiple Data Stream (MIMD)

Flynn Taxonomy



Instruction Stream: stream of instructions to execute.

Data Stream: stream of data to work on in the instruction execution.

[1]

- Single Instruction Stream / Single Data Stream (SISD)
- Single Instruction Stream / Multiple Data Stream (SIMD)
- Multiple Instruction Stream / Single Data Stream (MISD)
- Multiple Instruction Stream / Multiple Data Stream (MIMD)

Current Computer Systems

- Current Computing Systems are hybrid respect to the Flynn taxonomy, and often heterogeneous

HYBRID
Several levels of different kind of parallelism nested

HETEROGENEOUS
Subsystems with totally different architectures are put together to cooperate

SEVERAL combinations of Flynn's classes

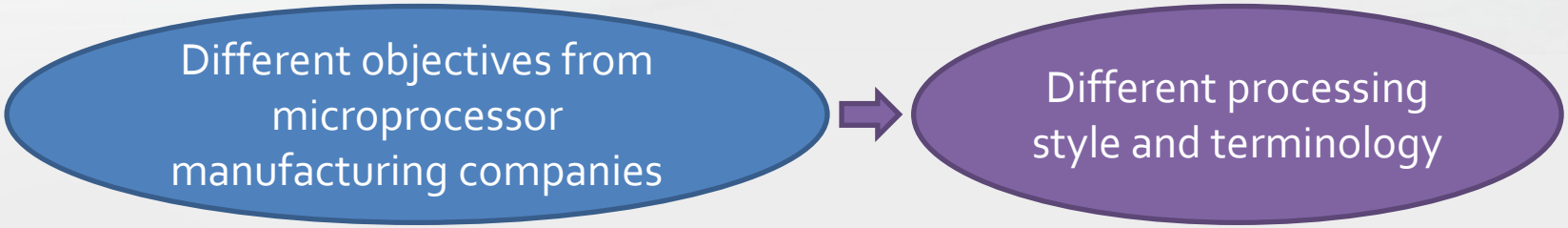
GPU
Evolution

- The video game industry has heavily invested in improving graphical processing.
  - In the early 1980s, specialized components for graphical rendering were introduced to lighten the computational load of CPUs.

Different objectives from microprocessor manufacturing companies → Different processing style and terminology

GPU
Evolution

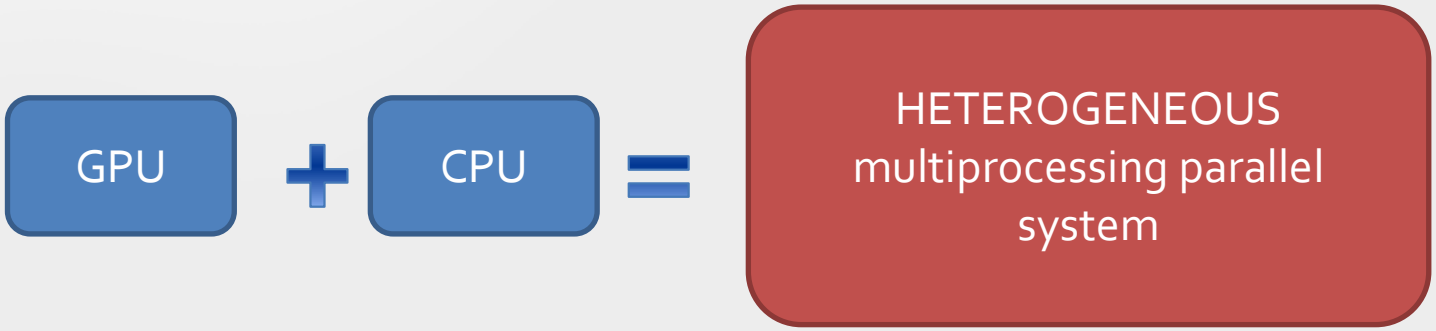
- The video game industry has heavily invested in improving graphical processing.
  - In the early 1980s, specialized components for graphical rendering were introduced to lighten the computational load of CPUs.



  - GRAPHICS PIPELINE: rendering process (generation of an image from a description of 3D objects) carried out through more or less specialized hardware.

GPU
Evolution

- GPUs, as coprocessors integrated with the CPU, do not need to perform all the tasks normally executed by a CPU.
  - They can dedicate all resources to graphics.
  - In a system that contains both a GPU and a CPU, the CPU will perform the tasks for which the GPU is not efficient



- Over time, they have become increasingly programmable and precise in calculations.

GPU
Evolution

- Modern devices are highly parallel and multithreaded multiprocessors that hide high memory latency by exploiting parallelism among many threads, so the main memory is optimized to maximize transfer bandwidth rather than minimize latency.

- They are now designed using identical general-purpose processors, making them more similar to the multicore architectures of conventional microprocessors.

Same type of core:
- Greater scalability
- Better load balancing

BETTER PERFORMANCE

GPU
Evolution

- Modern devices are highly parallel and multithreaded multiprocessors that hide high memory latency by exploiting parallelism among many threads, so the main memory is optimized to maximize transfer bandwidth rather than minimize latency.

- They are now designed using identical general-purpose processors, making them more similar to the multicore architectures of conventional microprocessors.

General Purpose GPUs
used for general applications

GPU
Evolution

- Modern devices are highly parallel and multithreaded multiprocessors that hide high memory latency by exploiting parallelism among many threads, so the main memory is optimized to maximize transfer bandwidth rather than minimize latency.

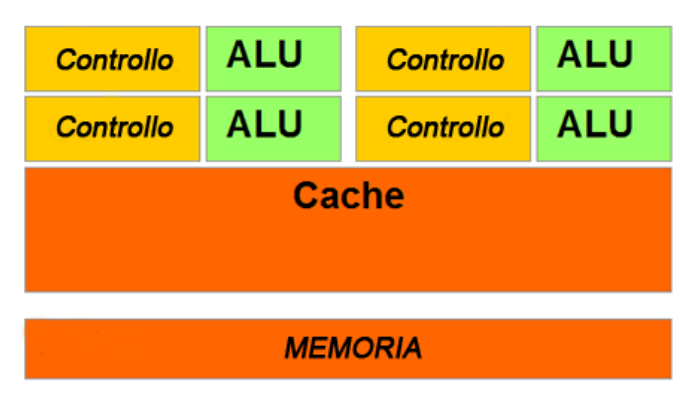
- They are now designed using identical general-purpose processors, making them more similar to the multicore architectures of conventional microprocessors.

Paired with the CPU which maintains control (hence the term 'accelerator')

GPU vs CPU

CPU

- A lot of on-chip space dedicated to control and cache
- Few processing graphics units, powerful and sophisticated
- Suitable for databases, recursive algorithms, irregular control flows



GPU

- Many units dedicated to computation
- Suitable for repetitive calculations and large amounts of data
- On-chip memory avoids the bottleneck of the system bus

GPU vs CPU

https://www.youtube.com/watch?v=-P28LKWTzrI

https://www.youtube.com/watch?v=ZrJeYFxpUyQ

Difference according to NVIDIA…

GPU vs CPU

In reality…

- Not for all problems does the superiority of the GPU hold true.
- Not for all WAYS of addressing problems does the superiority of the GPU hold true.
  - Even in the most modern architectures, flexibility does not reach that of CPUs.

- The really important thing is to recognize the best way to make the GPUs and CPUs of the system we have available work together.
  - GPUs are used to ACCELERATE portions of code with the right characteristics.
  - They are now available in almost all systems, from PCs to supercomputers, at relatively low costs.
  - The number of cores they provide continues to grow.

GPU
CUDA

- With GPGPU, high-level languages suitable for programming these devices are also emerging.

- At the end of the 2000s, NVIDIA introduced a specific parallel processing architecture: **CUDA** (Compute Unified Device Architecture).
  - With this architecture, extensions for the main programming languages, especially C, were also born, which identify a scalable parallel programming model with shared memory.
  - Through CUDA/C, the developer can use some virtualizations of the architecture that greatly simplify the programmability of the GPU.

- In 2008, AMD and NVIDIA, together with other industry companies, invested in the creation of OpenCL, a language that follows the philosophy and principles of CUDA/C but conceived as *open-source*.
  - Since 2009, OpenCL has been supported by the products of both companies.

GPU
CUDA

- Delving into CUDA and CUDA/C is a good way to approach the study of GPUs.
  - The NVIDIA G80 architecture marked a trend in design choices, not radically different from those made by AMD (ATI), which now characterize this type of device.
  - The CUDA model is also applicable to other parallel processing architectures with shared memory, such as multicore CPUs.
  - The abstractions introduced by CUDA exactly reflect the characteristics of NVIDIA GPU hardware, which combine multiple levels of parallelism: these abstractions guide the programmer in problem division and work synchronization.

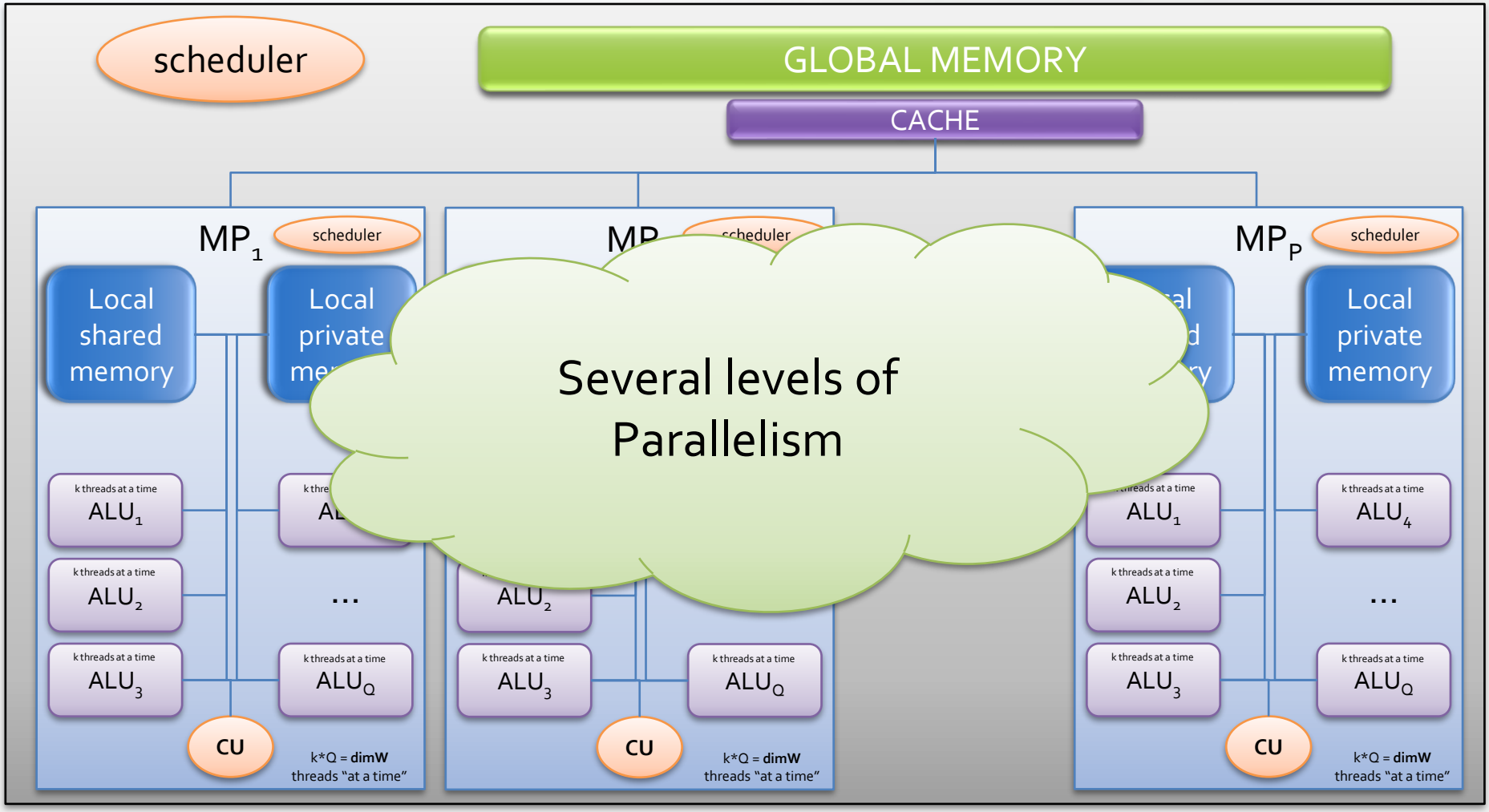
GPU

GPU Architecture

Let's now start modeling
an architecture that reflects the characteristics of the most common GPUs

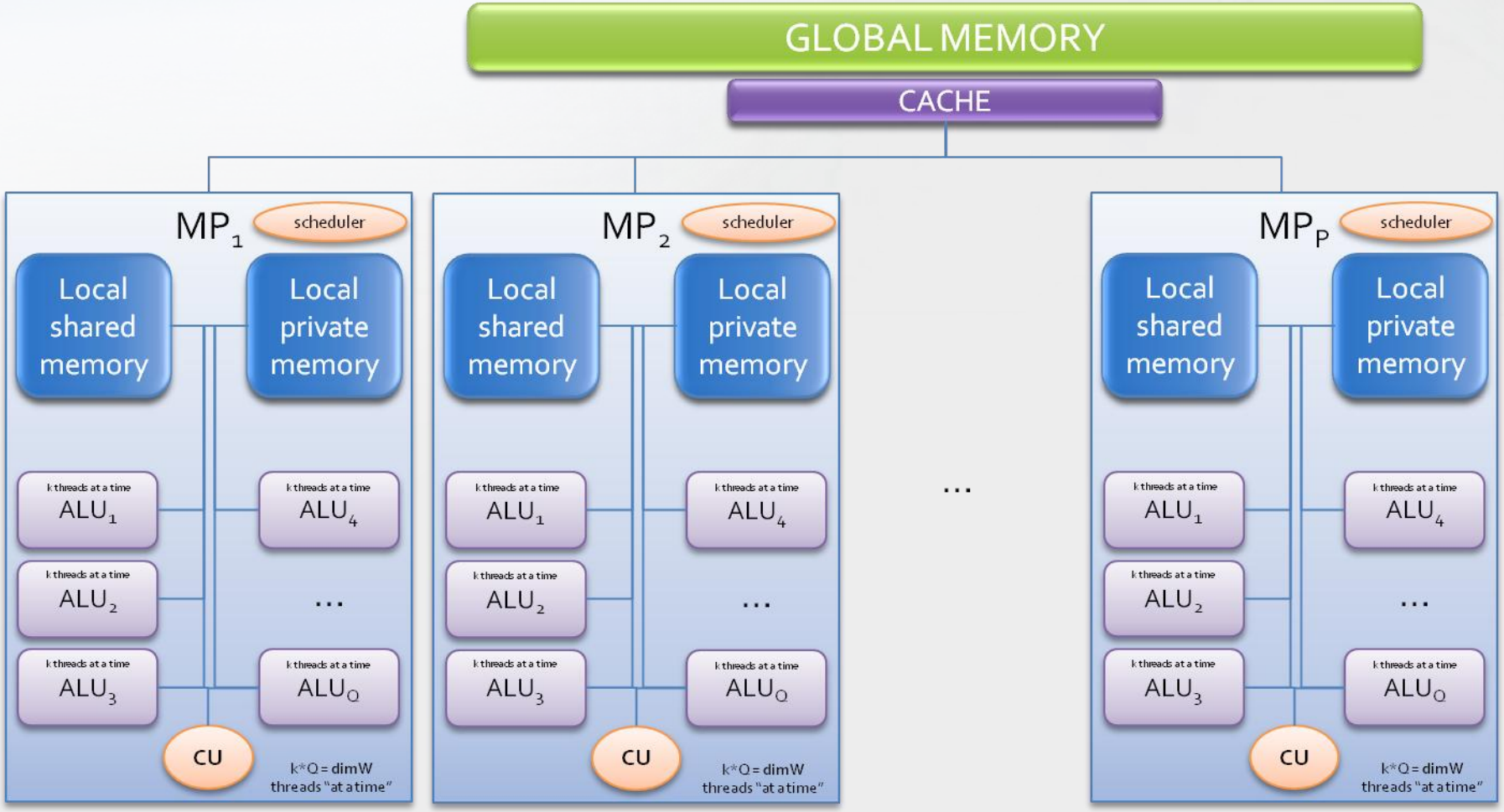
GPU

GPU Architecture: Several levels of Parallelism



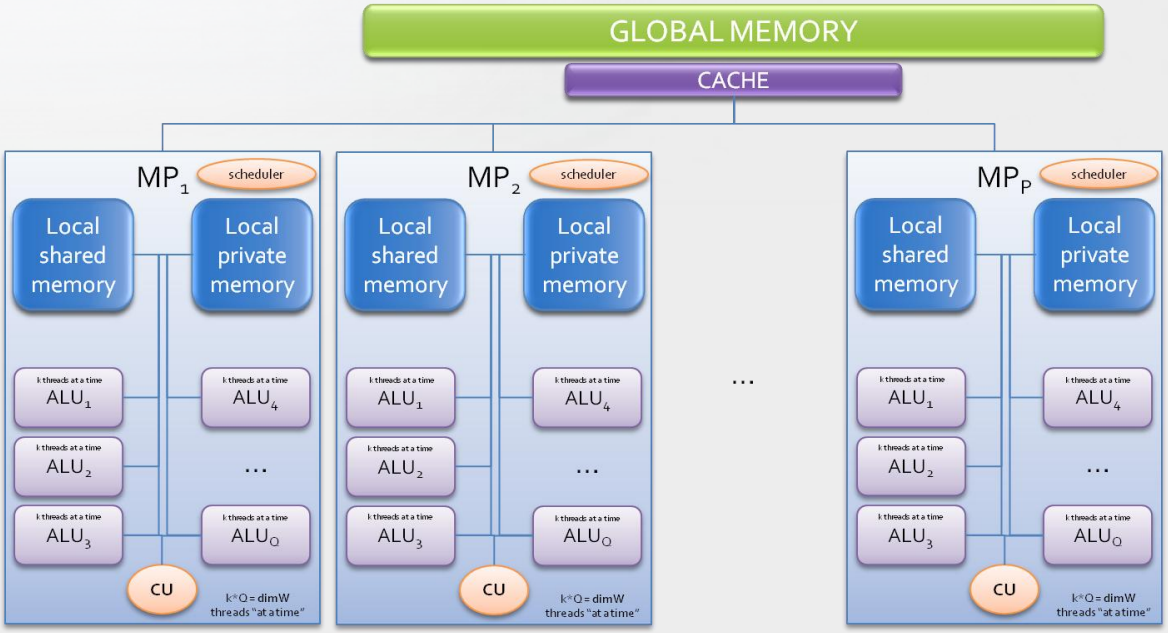
GPU

GPU Architecture: Several levels of Parallelism



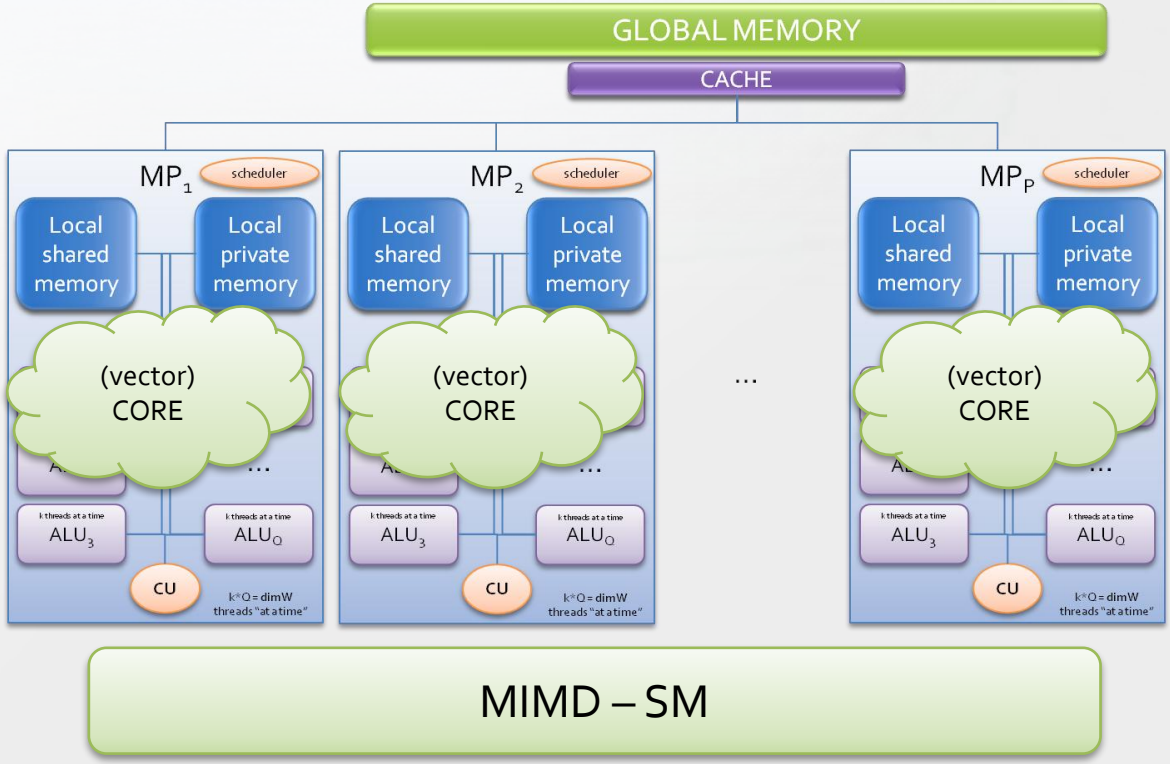
GPU

GPU Architecture: Several levels of Parallelism



GPU

GPU Architecture: Several levels of Parallelism



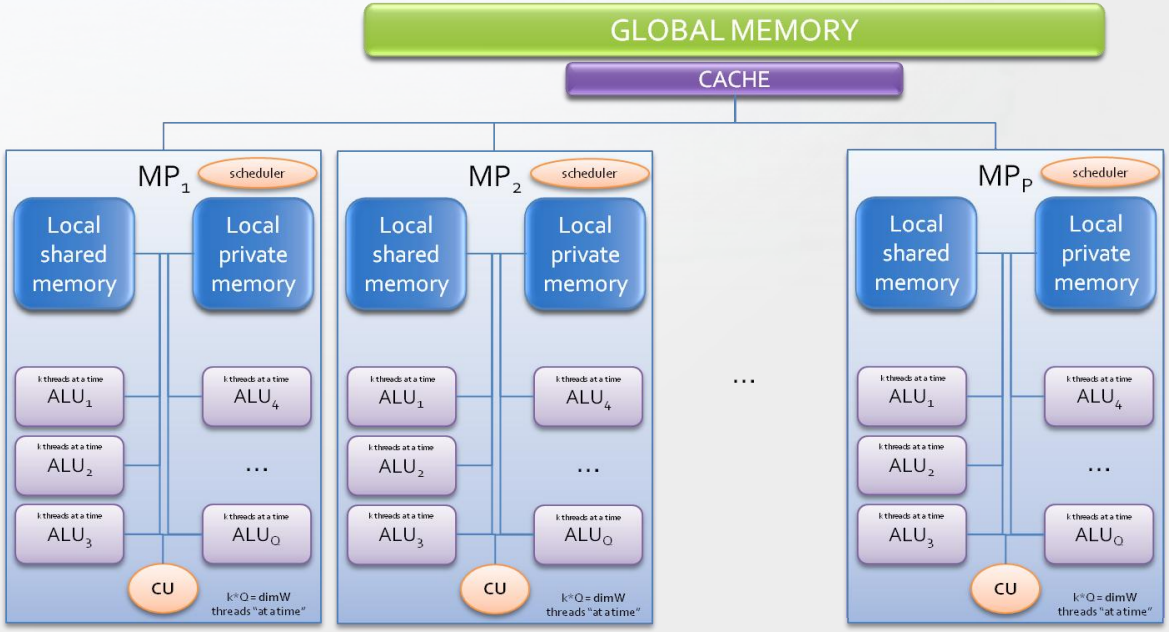
Highest level

P cores sharing a memory (with possible cache):

MIMD shared memory machine.

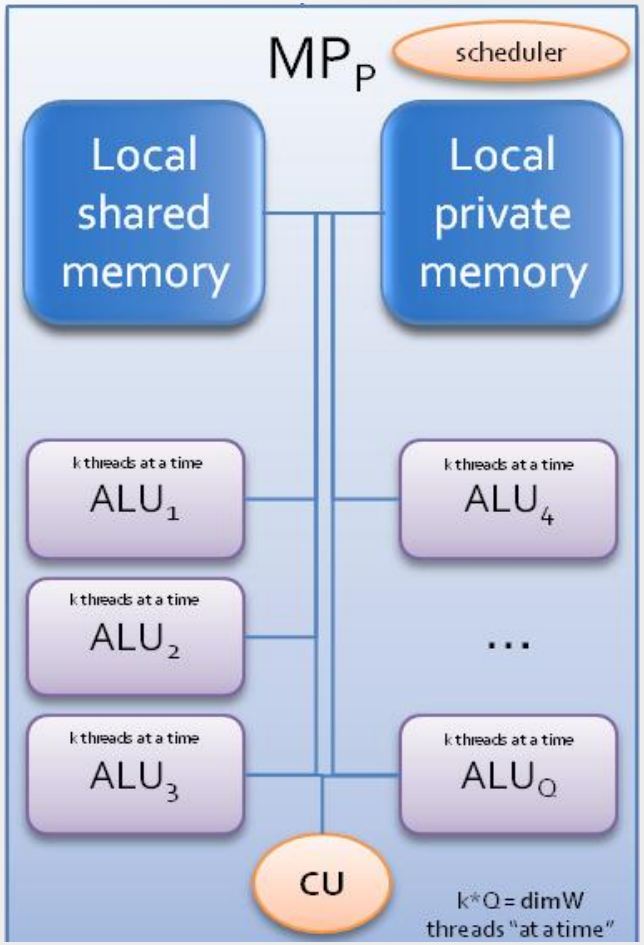
GPU

GPU Architecture: Several levels of Parallelism



GPU

GPU Architecture: Several levels of Parallelism



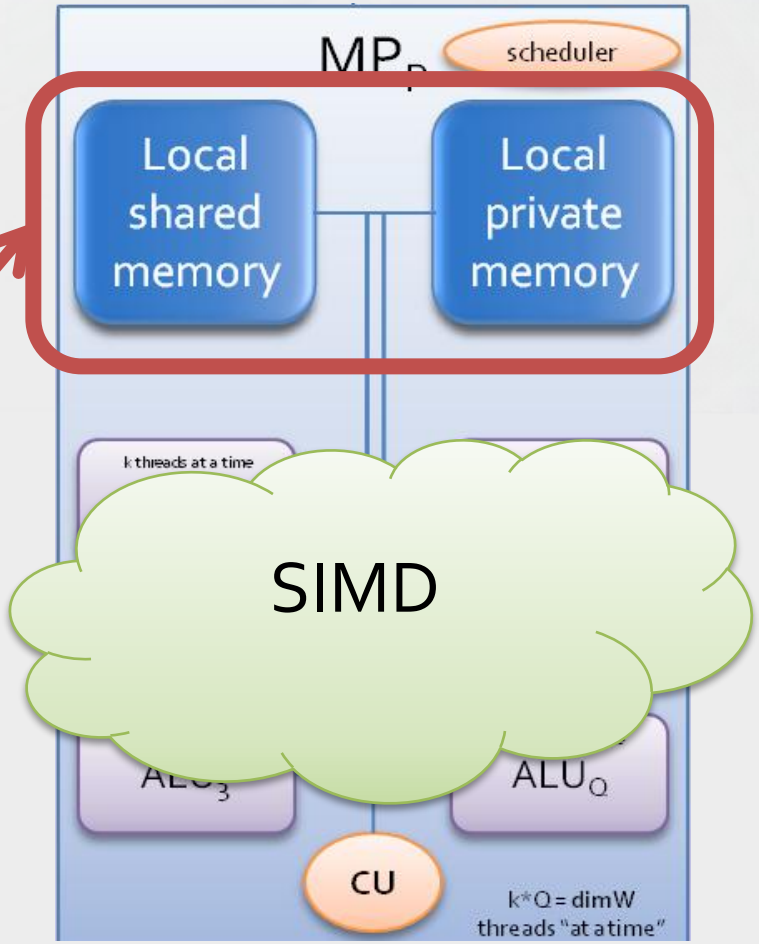
GPU

GPU Architecture: Several levels of Parallelism

Intermediate level

Each MP is a SIMD calculator with **Q** ALUs,

Each MP has its own local memory area



GPU

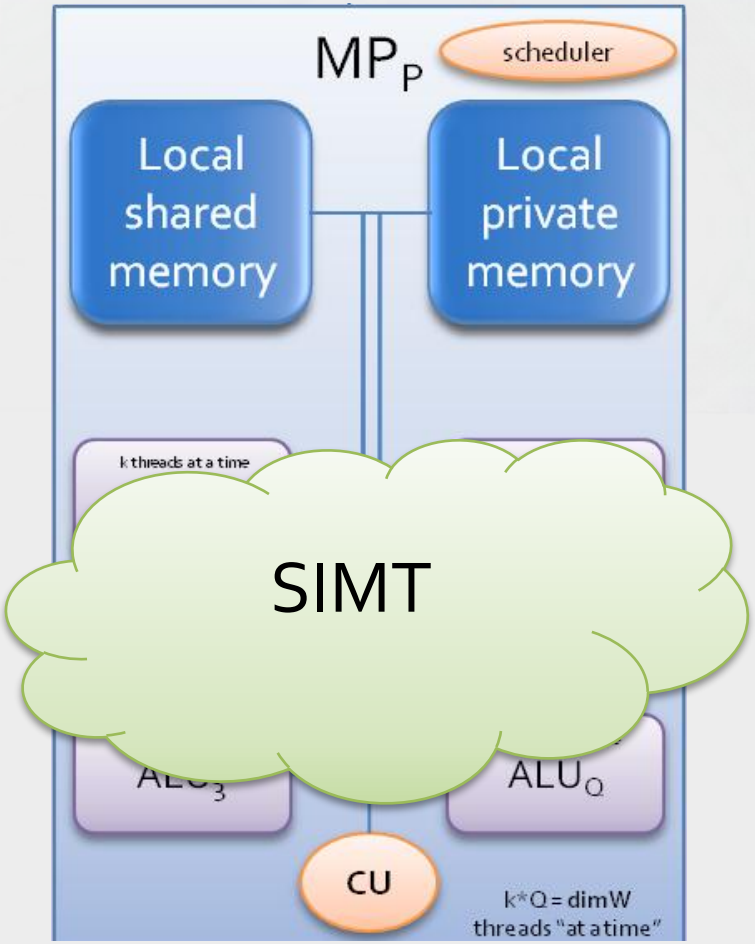
GPU Architecture: Several levels of Parallelism

Intermediate level

Each MP is a SIMD calculator with **Q** ALUs,

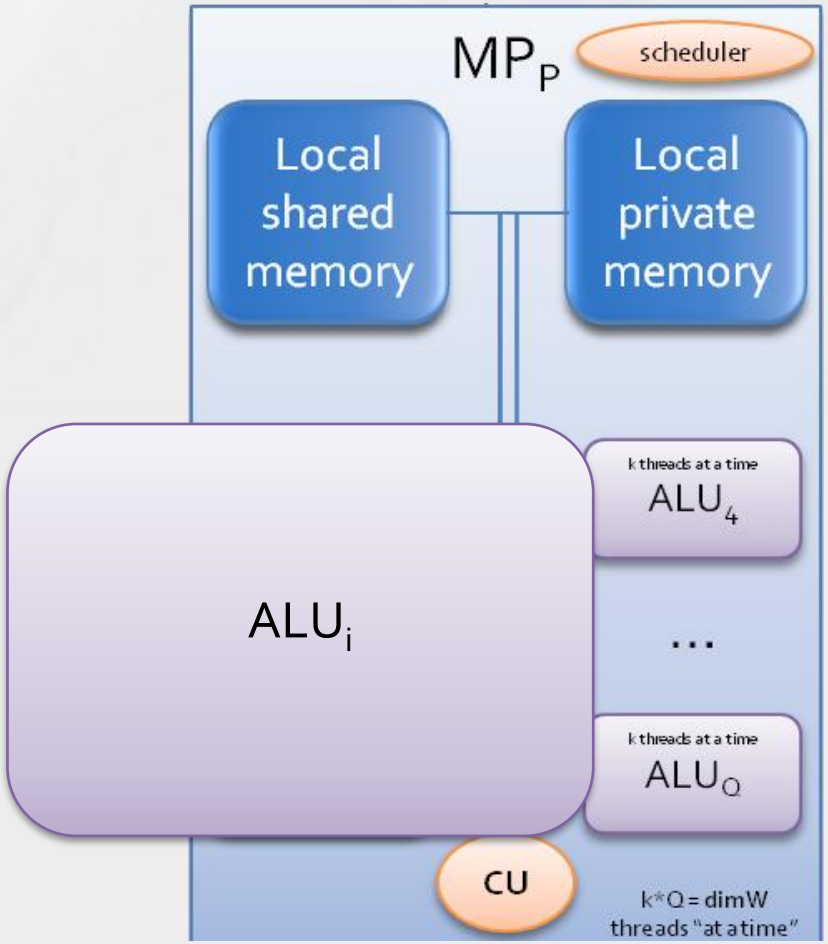
Commercially, it is called **SIMT**
That is
Single Instruction Multiple Threads

This is because in the physical implementation, they are not ALUs but small and simple complete CPUs, and the threads can have slightly different behaviors from each other (branches are allowed): they execute the same portion of code, but not always EXACTLY the same instructions.



GPU

GPU Architecture: Several levels of Parallelism

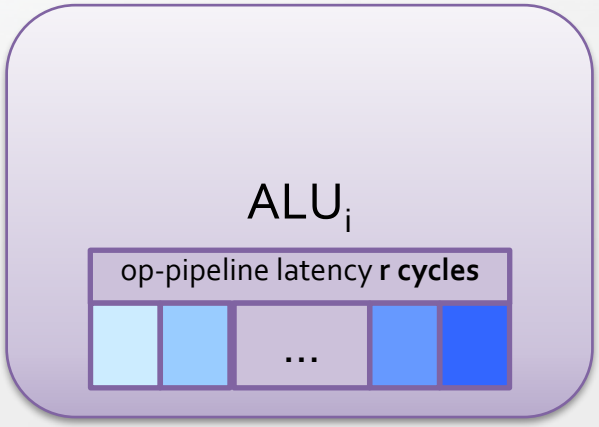


GPU

GPU Architecture: Several levels of Parallelism

GPU

GPU Architecture: Several levels of Parallelism

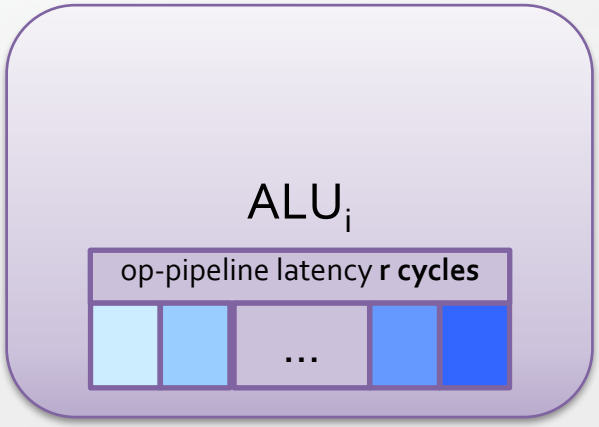


Lowest level

Each ALU is pipelined

GPU

GPU Architecture: Several levels of Parallelism



Lowest level

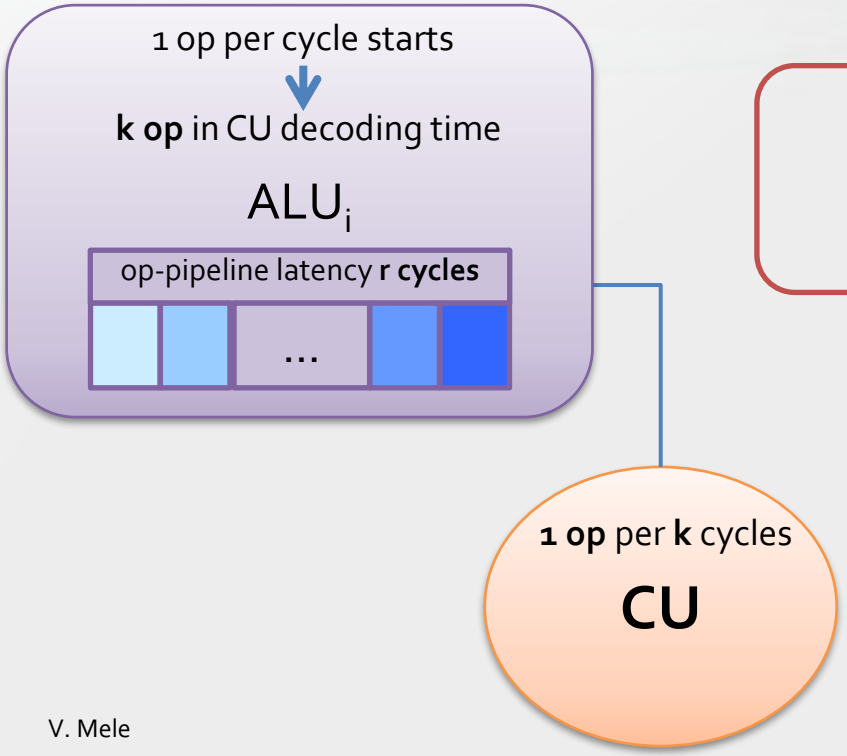
Each ALU is pipelined

pipeline latency of **r** cycles.

GPU

How many simultaneous threads?

The CU of the MP takes **k<r** cycles to decode an instruction.
(Let's consider that one instruction corresponds to one operation).



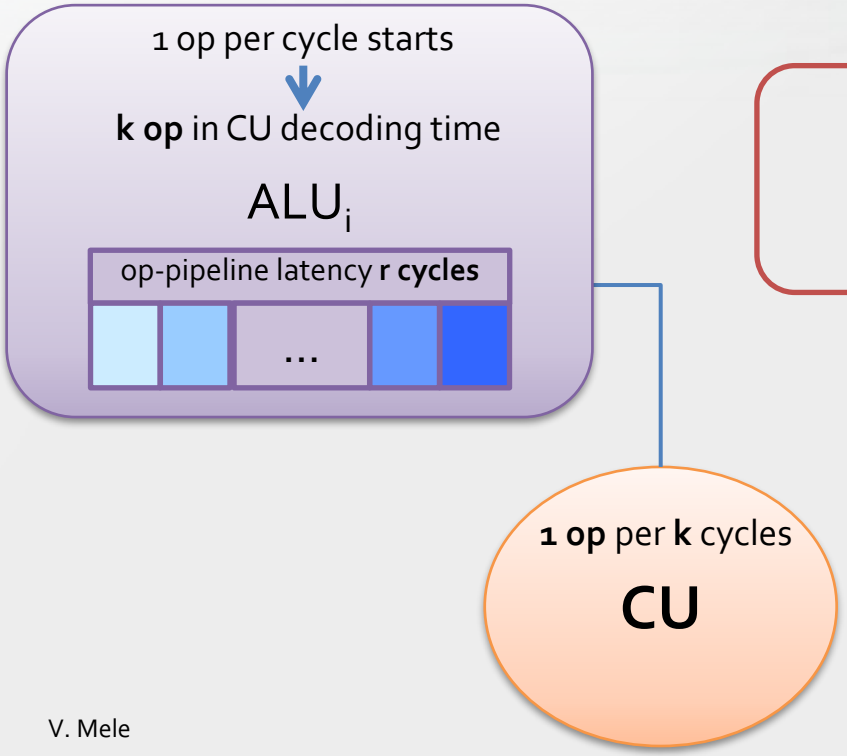
In the pipeline of each ALU,
1 operation can enter per cycle.

For each decoded instruction,
k operations can start.

GPU

How many simultaneous threads?

The CU of the MP takes **k<r** cycles to decode an instruction.
(Let's consider that one instruction corresponds to one operation).



In the pipeline of each ALU,
1 operation can enter per cycle.

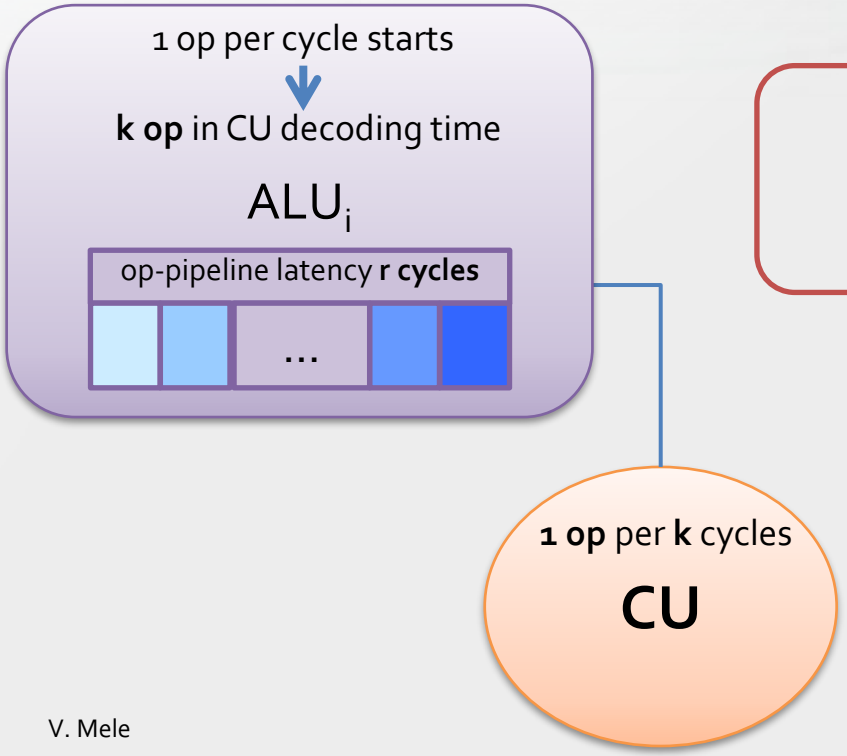
For each decoded instruction,
k operations can start.

GPU

How many simultaneous thr

If the CU takes k cycles to decode, every **k cycles** a different instruction enters the pipeline.
If one instruction can enter the pipeline each cycle, then between one decoding and another, k instructions already in the pipeline will exit (**finish**).
It is like having executed **k threads simultaneously**.

The CU of the MP takes **k<r** cycles to decode an instruction.
(Let's consider that one instruction corresponds to one operation).



In the pipeline of each ALU,
1 operation can enter per cycle.

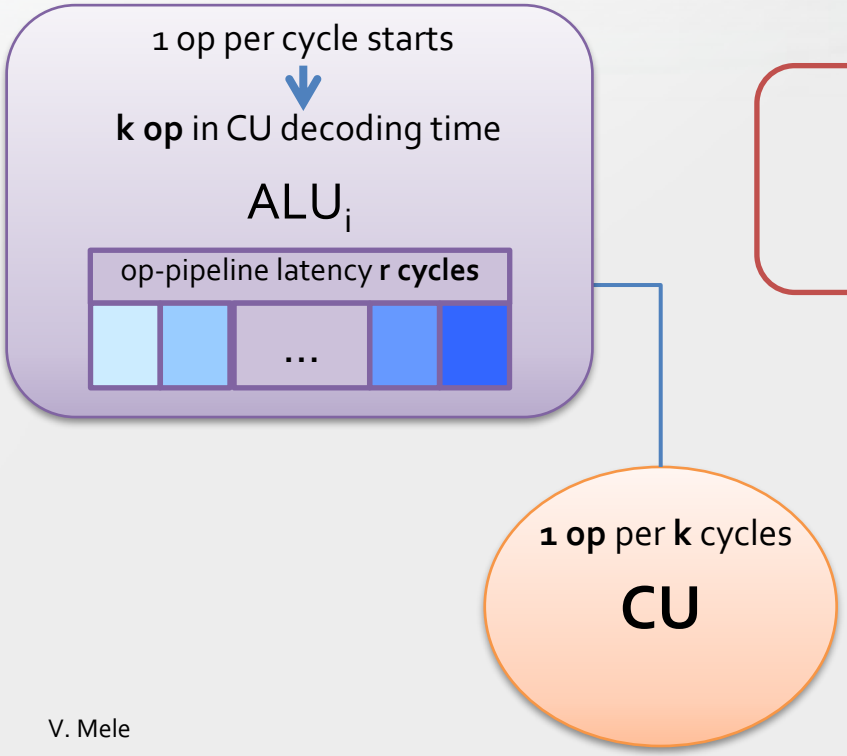
For each decoded instruction,
k operations can start.

GPU

How many simultaneous t

k is assumed to be less than r (the number of cycles required for the pipeline to reach steady state) because if the CU takes longer than the pipeline latency to decode, the pipeline never reaches steady state.

The CU of the MP takes **k** cycles to decode an instruction.
(Let's consider that one instruction corresponds to one operation).

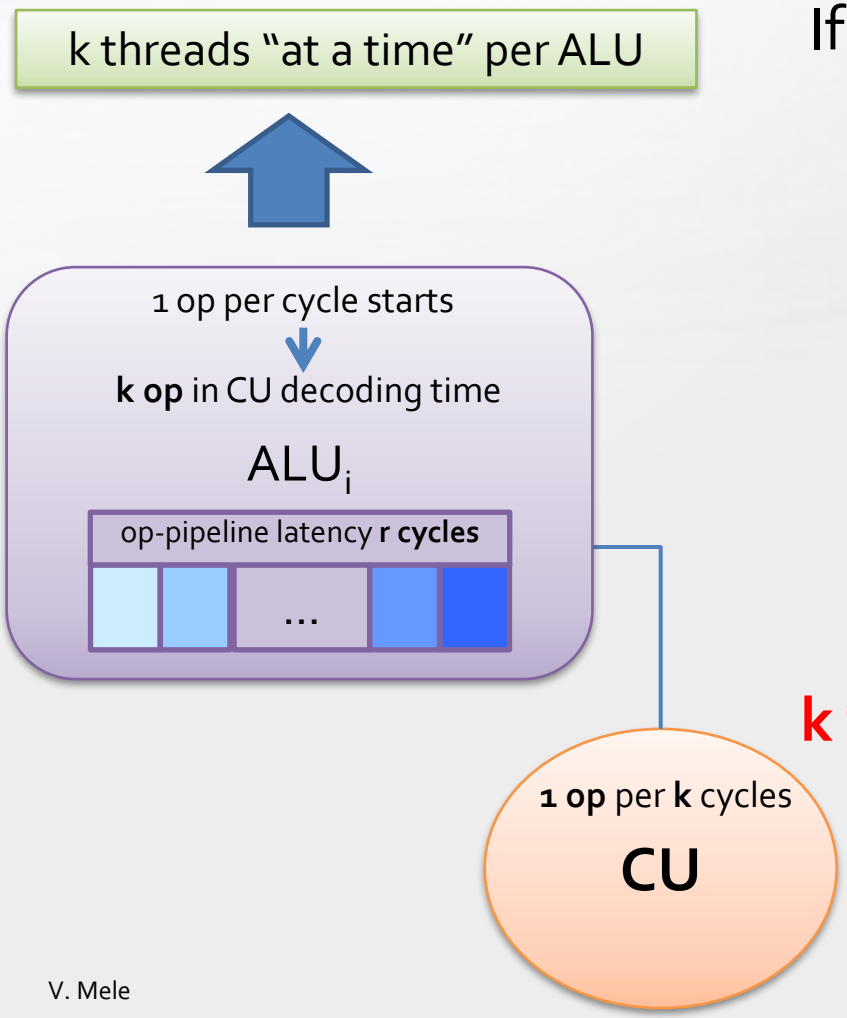


In the pipeline of each ALU,
1 operation can enter per cycle.

For each decoded instruction,
k operations can start.

GPU

How many simultaneous threads?



If we consider that each operation is performed by a different thread

1 operation → 1 thread

for each decoded instruction
k threads execute the corresponding operation
on each ALU

GPU

How many simultaneous threads?

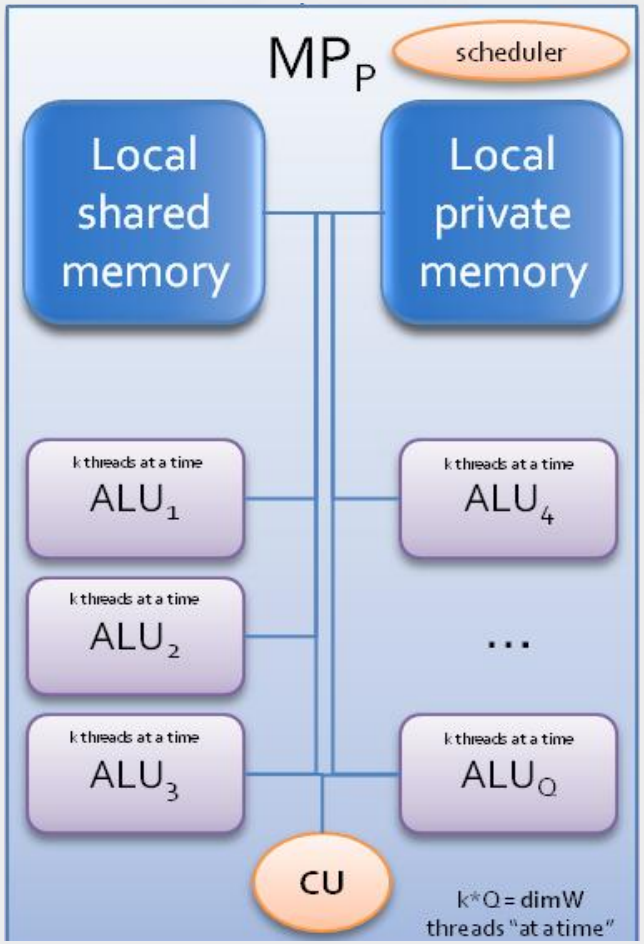
GPU

How many simultaneous threads?

Each MP has Q ALUs

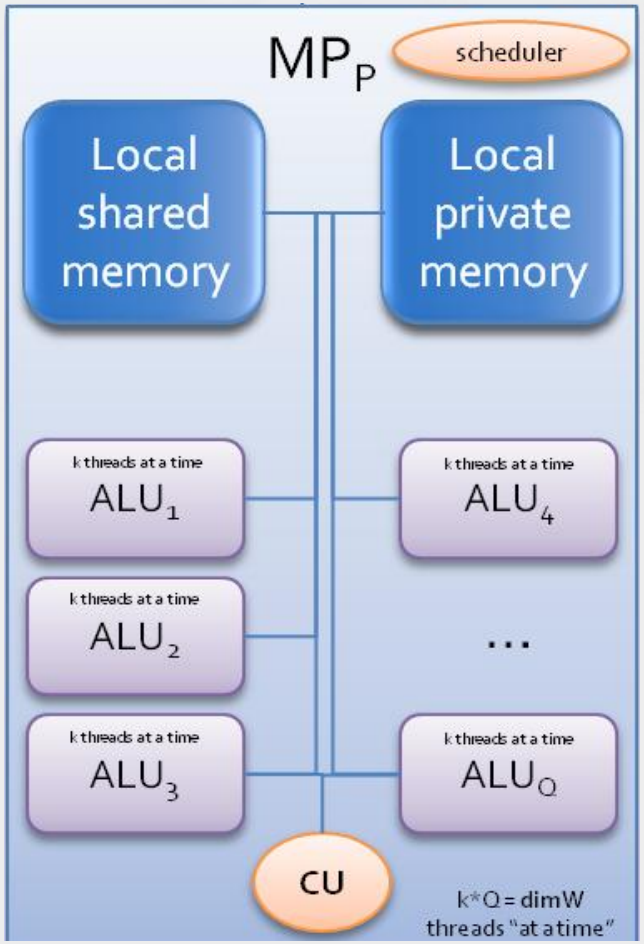
For each instruction on the MP
k*Q thread are started

Warp
group of **k*Q** thread simultaneously executed by an MP.
k*Q=dimW



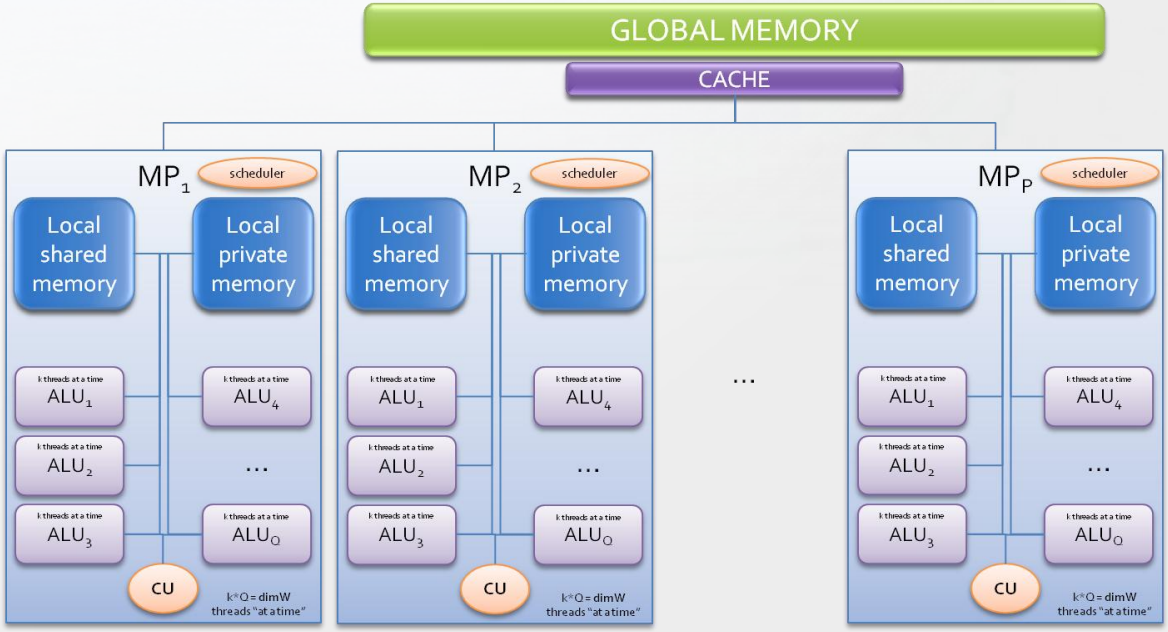
GPU

How many simultaneous threads?



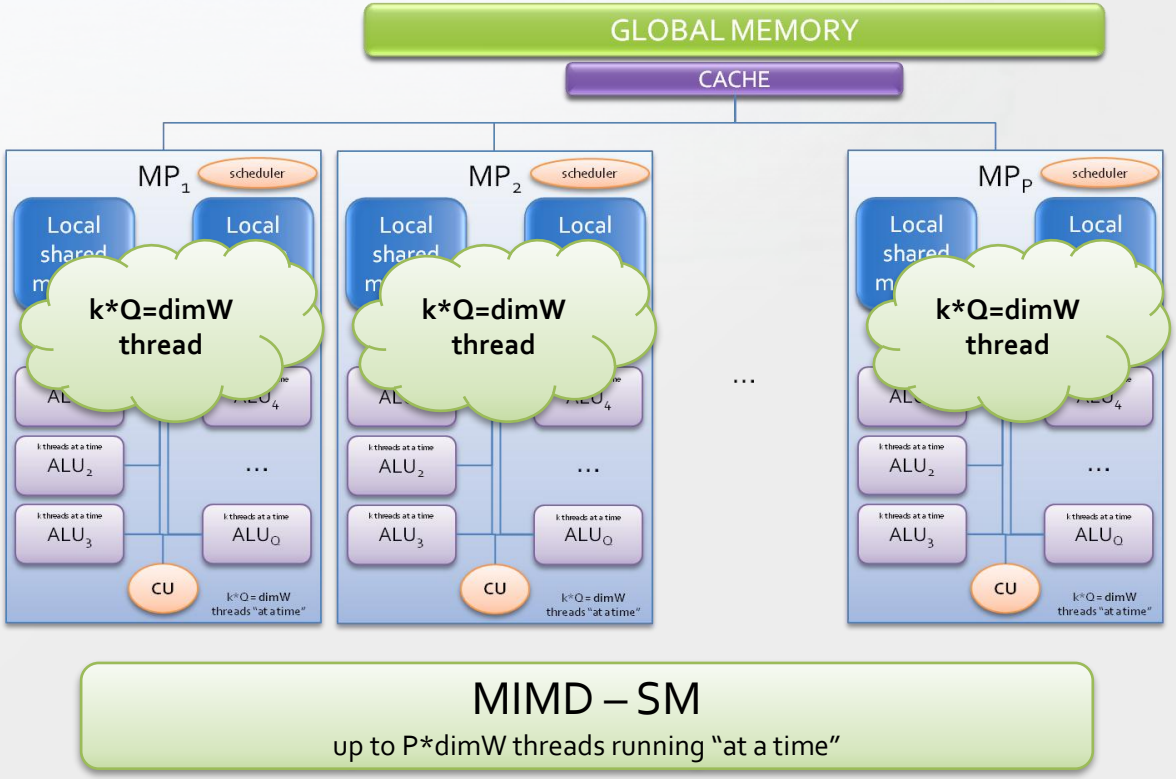
GPU

How many simultaneous threads?



GPU

How many simultaneous threads?



Each GPU has P MP

For each instruction on the GPU, up to **P*dimW=P*k*Q** threads are started simultaneously

GPU

Computation time

P cores with Q units
Pipeline with k segments
k threads for each unit

Each thread executes instructions that are all dependent on each other

Fully parallel algorithm
T sequential execution time

IDEAL!!

(Theoretical) Execution time of an algorithm for GPU

$$T_{GPU}(P \cdot Q \cdot k, N) = \frac{T(N)}{P \cdot Q \cdot k} \cdot t_{calc}$$

N problem size

This time must be added to **the memory access time**

GPU

Computation time

P cores with Q units
Pipeline with k segments
k threads for each unit

Each thread executes instructions that are all dependent on each other

Fully parallel algorithm
T sequential execution time

IDEAL!!

(Theoretical) Execution time of an algorithm for GPU

$$T_{GPU}(P \cdot Q \cdot k, N) = \frac{T(N)}{P \cdot Q \cdot k} \cdot t_{calc}$$

N problem size

By launching the execution of **<u>a number of threads much greater than P*Q*k</u>** (possibly a multiple) for the **same size N,** the theoretical computation time increases, but the total execution time **does NOT**: memory latency is covered.

GPU

Computation time

P cores with Q units
Pipeline with k segments
k threads for each unit

Each thread executes instructions that are all dependent on each other

Fully parallel algorithm
T sequential execution time

IDEAL!!

(Theoretical) Execution time of an algorithm for GPU

$$T_{GPU}(P \cdot Q \cdot k, N) = \frac{T(N)}{P \cdot Q \cdot k} \cdot t_{calc}$$

N problem size

We solve a larger problem in the same execution time.
That is,
the **scalability** can be greater than what one would expect from the number of units.

GPU

Computation time

P cores with Q units
Pipeline with k segments
k threads for each unit

Each thread executes instructions that are all dependent on each other

Fully parallel algorithm
T sequential execution time

IDEAL!!

(Theoretical) Execution time of an algorithm for GPU

$$T_{GPU}(P \cdot Q \cdot k, N) = \frac{T(N)}{P \cdot Q \cdot k} \cdot t_{calc}$$

N problem size

FUNDAMENTAL ASSUMPTION: Perfectly parallel algorithm

The load must be very well balanced among the units: excellent performance in applications that involve the same operations on different data (such as rendering).